

The Non-Programmer's Guide to Building Software with Claude Code (Battle Tested)

By Frank Nillard



What is a "Project Harness"?

A project harness is the set of files and conventions that give Claude Code persistent context about your project across sessions. Without it, every conversation starts from scratch. With it, Claude understands your product, your architecture, your design system, and your preferences from the first message.

The harness has 5 layers:

1. **Global preferences:** Who you are, how you work.
2. **Project instructions:** What you're building, how it works.
3. **Persistent memory:** Business context, corrections, lessons learned.
4. **Specialist agents:** Domain experts with baked-in knowledge.
5. **Automated guardrails:** Hooks that catch mistakes automatically.

The Files You Need

This section details the core files needed for an effective project harness, ensuring Claude Code has all the necessary context to assist you efficiently.

2.1 - Global **CLAUDE.md** (`~/claude/CLAUDE.md`)

This file houses your personal preferences that apply to **all** projects. Think of it as your style guide for Claude.

Include sections on:

- Code quality principles
- Communication style
- Decision-making framework
- Which MCP servers to use and when

Example structure:

2.2 - Project CLAUDE.md (your-project/CLAUDE.md)

This is the **master config file** for your specific project. It provides Claude with crucial information about the project's structure and conventions.

What to include:

- Tech stack
- Commands (dev/build/lint/test)
- Architecture overview
- Auth patterns
- Database patterns
- Deployment info
- Gotchas/pitfalls

Use references to link to other docs so Claude loads them all.

Example sections:

2.3 - Product Requirements Document (docs/PRD.md)

This document should contain the **full product vision** AND the **MVP scope** in one place.

Include:

- What the product is
- User types
- All pages/features (full vision)
- MVP subset
- Data models (SQL schemas)
- Payment flows
- Auth flows
- "Done" definitions per page
- What's explicitly out of scope

Why this matters: Claude needs to understand the **WHOLE** product to make good decisions about **any** single feature.

2.4 - Design System Document (DESIGN.md)

This document outlines your visual language, ensuring consistency across your application.

Include:

- Visual language
- Color tokens
- Typography
- Component patterns
- Rules and constraints (e.g., "never use borders, only background color shifts")
- Do's and Don'ts with examples

- Component code snippets Claude can copy

2.5 - Activity Log (activity.md)

This is a session-by-session decision journal, crucial for providing historical context to Claude.

Include:

- What was completed
- What bugs were found
- What lessons were learned
- What decisions were made

This gives Claude historical context — it knows **WHY** things are the way they are.

Format:

Example:

2.6 - Scratchpad (docs/scratchpad.md)

This file serves as a central location for strategy discussions, open questions, and, most importantly, the **BUILD CHECKLIST**.

Rules:

- Never create separate TODO lists
- Mark items with session number when done
- New tasks go into appropriate batches

2.7 - Route Map (docs/ROUTES.md)

This document provides a comprehensive overview of every route in your application.

Include:

- Every route in the app
- Its build status (BUILT/STUB/WIRED/MISSING)
- How users reach it
- User flow sequences showing route-to-route navigation
- Known issues table

2.8 - User Flows (docs/USER-FLOWS.md)

Detail step-by-step scenarios for each user type in this document.

Include:

- Step-by-step scenarios for each user type
- State matrix showing what happens in each combination of conditions
- What's implemented vs. what's TODO

Specialist Agents

Agents are like hiring domain experts who already know your codebase. They live in `agents` as markdown files.

Structure of an agent file:

- Frontmatter: name, description, tools, model
- System prompt: domain knowledge, rules, code patterns, what NOT to do

Recommended starter agents for a web app:

- Database agent — knows your full schema, RLS rules, migration patterns
- UI agent — knows your design system, component patterns, color tokens
- Auth agent — knows your auth flow, middleware patterns, session management
- Payments agent — knows your payment flows, webhook handlers, fee calculations
- QA agent (read-only) — reviews implementations for spec compliance, security, edge cases

Key principles:

- Bake domain knowledge **INTO** the agent prompt (schemas, patterns, rules)
- Agents shouldn't need to explore — they should already know
- QA agent should be read-only (no Edit/Write tools) so it can't "fix" what it finds
- Use Sonnet model for agents (fast, capable, cost-effective)

The Memory System

Memory persists across conversations. It stores things Claude can't derive from reading code.

4 types of memory:

- **User:** Your role, expertise level, preferences.
- **Feedback:** Corrections you've given Claude (what to stop doing, what to keep doing).
- **Project:** Business context, strategy decisions, stakeholder info.
- **Reference:** Pointers to external resources (project trackers, dashboards, etc.).

What **NOT** to save:

- Code patterns (they're in the code)
- Git history (use git log)
- Debugging solutions (they're in commits)
- Anything in `CLAUDE.md` already

Memory files go in `memory` with a `index`.

Hooks — Automated Guardrails

Hooks are shell commands that run automatically at specific points in Claude's workflow. They catch mistakes before they happen.

Essential hooks for non-programmers:

1. **SessionStart hook:** Loads project context (branch, node version) automatically.
2. **Pre-Bash hook:** Blocks dangerous commands (, etc.), prompts before installing production dependencies.
3. **Pre-Write/Edit hook:** Scans for hardcoded secrets/API keys before they get written to files.
4. **Post-Write/Edit hooks (async):** Runs linter, type checker, and tests automatically after every file change.
5. **Stop hook:** Blocks session exit if you have uncommitted changes (prevents losing work).
6. **Notification hook:** Plays a sound when Claude needs your permission.

How to set them up: edit , add hooks under the key.

Example:

The Workflow — Brainstorm → Spec → Plan → Build → Review

This section outlines a development workflow optimized for non-programmers, ensuring quality through process.

Step 1: **BRAINSTORM** → Talk through what you want to build. Claude asks clarifying questions. Output: a design spec.

Step 2: **SPEC** → Written document capturing what, why, constraints, and decisions. Saved to .

Step 3: **PLAN** → Step-by-step implementation plan with file maps, task breakdowns, and code patterns. Saved to .

Step 4: **BUILD** → Execute the plan task-by-task. Use specialist agents for domain work. Mark tasks complete as you go.

Step 5: **REVIEW** → QA agent reviews the implementation. Check spec compliance, security, edge cases.

Step 6: **DEPLOY** → Push to production. Update with what happened.

- For production, I personally use Railway instead of Vercel: long term it will save you on costs.
- For storage, supabase is fine. RLS in built and a lot of other advantages
- For Auth, dont use Clerk (theyll charge you a hefty amount once you start scaling and get comfortable with it. They also make switching providers unnecessarily hard. Use Supabase Auth.

Why this matters for non-programmers: You can't review code line-by-line, so the structured workflow ensures quality through process. The spec catches requirement gaps. The plan catches architectural mistakes. The QA review catches implementation bugs.

Task Management Conventions

Rules:

- One single checklist in: **never** create separate TODO lists
- Organize work into batches (logical groupings of related tasks)
- Mark items with session number when done
- Logs the **WHY** (decisions, bugs, lessons)
- Tracks the **WHAT** (tasks, status, strategy)

Quick Start Checklist

A step-by-step checklist for setting up a new project:

- ☐ Create at project root with: tech stack, commands, architecture, links to docs
- ☐ Write with full vision + MVP scope + data models
- ☐ Write with design tokens, component patterns, rules
- ☐ Create for session logging
- ☐ Create with strategy + build checklist
- ☐ Create listing all routes and their status
- ☐ Create with step-by-step scenarios
- ☐ Create specialist agents in (database, ui, auth, payments, qa)
- ☐ Set up memory directory with index
- ☐ Configure hooks in (dangerous command blocker, secret scanner, linter, uncommitted changes blocker)
- ☐ Write your global at with personal preferences

Tips for Non-Programmers

- Write your PRD in plain language: Claude translates it to code
- The design system doc is your biggest leverage: it prevents every page from looking different
- Activity log is your insurance policy: when things break, the history tells you why
- Don't skip the QA agent: it catches things you can't see in the code
- Hooks are your safety net: they catch secrets, dangerous commands, and uncommitted work automatically
- Memory captures corrections: if you tell Claude to stop doing something, save it as feedback memory so it sticks across sessions
- Batch your work: small, focused batches ship faster than trying to build everything at once
- Always have a single source of truth for tasks: scattered TODO lists cause confusion
- Spec before you build: the 20 minutes spent writing a spec saves hours of rework
- Update activity.md every session: future you (and future Claude) will thank you

Next Steps

Now that you've established your project harness, you're ready to start building! Refer to the Quick Start Checklist to ensure you haven't missed any crucial steps. Remember to keep your documentation up-to-date and leverage the power of specialist agents and automated hooks for a smoother development experience.